

Repair Only When Evidence Demands It: Versioned Evidence Graphs for Cost-Aware Program Repair

ANONYMOUS AUTHOR(S)

Repository-level repair agents repeatedly inspect source code, recover executable reproductions, and validate candidate patches. A shorter interaction can be cheaper, but it can also stop before collecting evidence needed to distinguish a repair from an overfitting patch. We investigate whether the need for further interaction can instead be expressed as an explicit, versioned evidence question. We present ReachPatch, a repair controller that maintains requirements, program locations, observations, candidate patches, and outstanding questions in one evidence graph. A deterministic scheduler selects repair, evidence recovery, or falsification actions from unresolved questions; recorded dependencies determine when a question must be reconsidered. Exact evidence identities suppress repeated actions without treating missing evidence as success. Our empirical audit identifies an important accounting sensitivity: a 20-issue retained-attempt pilot suggests a 29.3% token reduction, whereas the completed 49-issue historical comparison records 21.45 versus 16.35 million tokens when archived attempts are included. Unequal retries and incomplete records preclude a causal interpretation of either contrast. We therefore separate this retrospective evidence from a prospective, blocked factorial study with durable request accounting and independent behavioral checks. The central research question is whether explicit evidence state can reduce the work requested of repair agents without abandoning necessary investigation; the current repository-level evidence does not yet establish that claim.

CCS Concepts: • **Software and its engineering** → **Software verification and validation**; *Software evolution*.

Additional Key Words and Phrases: automated program repair, evidence reuse, agent efficiency, demand-driven validation

1 Introduction

The cost of a repair agent is not determined solely by the size of its final patch. The agent may repeatedly inspect the same implementation, reconstruct a reproduction that already exists, or reconsider an observation whose relevant context has not changed. Each interaction also carries some of the preceding trajectory into another model request. Recent work demonstrates that this accumulated context contains removable waste: AgentDiet reduces trajectories during execution while accounting for the cost of the reduction model itself [8]. Other work improves the granularity of debugging interactions [7] or assigns different editing subtasks to different-sized models [5]. These findings make efficiency a substantive program-repair research problem, rather than merely an implementation concern.

An additional question arises *before* an interaction is issued: what unresolved fact would justify doing this work again? An agent may have already read a source span, yet a subsequent patch can make that read stale. A reproduction may execute successfully, yet establish nothing about the behavior requested in the issue. Conversely, the relevant evidence may be unchanged, making another recovery round unnecessary. Neither a fixed repair–test alternation nor indiscriminate memoization distinguishes these situations. The first repeats work; the second risks retaining conclusions after their supporting evidence changes.

We study this distinction through *versioned evidence questions*. A question identifies an unresolved claim about a candidate patch, the evidence on which its answer depends, and the conditions under which that answer must be revisited. For example, “does the reported operation satisfy the stated boundary behavior?” differs from “can the reproduction import the intended module?” The former may justify a repair or a counterexample; the latter justifies resolving an execution

prerequisite. Repeating either action without a change in its question or evidence is a candidate for suppression. Importantly, an unanswered or blocked question is not a passing check.

ReachPatch stores these questions in the same graph as source locations, observations, validation obligations, and patch checkpoints. This is an operational evidence graph, not a separate search tree augmented with graph-shaped prompt context. The controller queries the graph to select an action, records what the action established, and revisits questions when their recorded versions change. Repair proposes a code change; falsification challenges a specific outstanding claim. Deterministic discovery and execution precede model-based recovery where the necessary operation is already known. The model remains responsible for constructing repairs and probes that cannot be obtained by those deterministic steps.

Our intended objective is lower end-to-end cost subject to retained repair effectiveness. It is not more candidate patches, more graph nodes, or a lower token count obtained by refusing difficult issues. It is also not a claim that the repair and falsification roles reach a game-theoretic equilibrium: the implemented scheduler uses deterministic evidence priorities, not a learned utility function or an equilibrium solver.

This manuscript makes three bounded contributions. First, it formulates repair interaction as scheduling versioned evidence questions and identifies the conditions under which reuse is admissible. Second, it describes a concrete controller that combines question-driven scheduling, exact reuse, and explicit stopping states within one graph. Third, it audits the difference between retained-attempt savings and all-attempt expenditure, and operationalizes a controlled study to distinguish scheduling benefits from premature stopping. The historical evidence does not establish an effectiveness–cost advantage: lower retained-attempt cost alone cannot distinguish avoided work from abandoned or retried work.

2 Motivation and Research Objective

2.1 Repeated work is state-dependent

Consider the illustrative implementation in Listing 1. The issue requires a missing limit to select a default, while an explicit zero must remain zero. A nearby public test also requires negative limits to be rejected. The example is constructed to explain the mechanism; it is not presented as a measured benchmark outcome.

Listing 1. Illustrative boundary bug with distinct target and preservation questions.

```
def normalize_limit(limit):
    if not limit:
        return 10
    if limit < 0:
        raise ValueError("negative limit")
    return limit
```

The initial read identifies the truthiness predicate. A reproduction of `normalize_limit(0)` returns 10 and supplies a stable target failure when repeated. If the agent asks for the identical source range again before editing it, the recorded source hash can identify an exact read reuse opportunity. If it changes the predicate to `limit is None`, however, the earlier source observation is stale. The target question must be re-evaluated for the new patch.

A repair that also deletes the negative-limit guard creates a different unresolved question: has the patch preserved the public rejection behavior? Repeating target recovery does not answer it. A targeted negative-input check does. If that check fails, the next repair should preserve the zero-input success while restoring rejection, rather than discard both changes. If a generated input has

no grounded expected behavior, executing it may still reveal a path or exception, but its result cannot be promoted to a trusted pass or regression.

This example separates three identities that are often conflated: the identity of a *request*, the identity of its *evidence context*, and the identity of the *claim* being investigated. Identical commands can be useful after an edit. Different commands can redundantly investigate the same unresolved claim. Our implementation suppresses only supported exact identities; it does not solve general semantic equivalence between commands or hypotheses.

2.2 Objective and scope

For a fixed issue distribution, let Y_i^π indicate whether the single patch selected by policy π resolves issue i under held-out evaluation. Let C_i^π include all model requests and retries needed to produce that patch. The intended objective is

$$\min_{\pi} \mathbb{E}[C_i^\pi] \quad \text{subject to} \quad \mathbb{E}[Y_i^\pi - Y_i^{\pi_0}] \geq -\delta, \quad (1)$$

where π_0 is a matched baseline and δ is a non-inferiority margin fixed before evaluation. This expression defines an experimental objective, not an optimization problem solved by our controller. Tokens, billed cost, model calls, and elapsed time are reported separately: provider caching and concurrency prevent them from being interchangeable.

The controller observes the issue description, the base repository, accessible public tests, and its own executions, following the repository-level issue setting of SWE-bench [2]. It does not observe the benchmark’s hidden tests or reference repair during generation. It returns one full base-to-selected-patch diff. An internally certified patch satisfies the available executable obligations; it is not a proof that the issue is fully repaired. Accordingly, internal certification and externally measured Resolved@1 are different outcomes throughout this paper.

3 Versioned Evidence as Repair State

3.1 One graph, multiple decision views

At step t , the controller maintains a typed graph $G_t = (V_t, E_t)$. The graph connects a candidate patch to the claims it must satisfy and to the observations that bear on those claims. Its essential relation is not simply that two entities are adjacent, but that a decision depends on a particular piece of evidence. For example, a checkpoint p requires validation of a question q ; q derives from requirement r and source evidence d ; and an execution observation o supports or refutes the claim in q . These links preserve the path from a repair decision back to its justification.

Two graph views are central to this dependency semantics. Let $\mathcal{Q}(G, p)$ denote the questions associated with checkpoint p , and let $\mathcal{D}(G, q)$ denote the dependencies recorded for question q . The scheduler operates on the open-question frontier

$$\mathcal{F}_Q(G) = \{(p, q) \mid p \in \mathcal{P}_{\text{open}}(G), q \in \mathcal{Q}(G, p), \text{status}_G(q) = \text{OPEN}\}, \quad (2)$$

where $\mathcal{P}_{\text{open}}(G)$ contains checkpoints still eligible for investigation. The dependency view determines the recorded state used to version each question. Together, these views let the controller ask both *what remains unresolved?* and *has the evidence for an earlier answer changed?* Figure 1 illustrates the connection. Neither view creates a second graph.

Program and execution relations refine the evidence attached to these questions. Static calls and approximate def-use identify possible dependencies; observed calls, branch outcomes, and failure locations record what an execution actually reached. The distinction prevents a possible path from becoming evidence that a requirement was exercised. Modification and localization links associate

a hypothesis with the source it proposes to change, while closure and regression links retain the outcomes of evaluating that change.

Program evidence is seeded before initial patch generation from issue symbols, source hints, public checks, and bounded local source analysis. Executions then add observations and dynamic paths. The implementation limits source exploration by files, symbols, edges, depth, and time; budget boundaries remain explicit frontiers. A missing edge at the graph budget must not silently remove an executable validation obligation: the validation view also examines applicable obligation nodes. The graph therefore records both what is known and where exploration stopped.

Each checkpoint contains the complete diff from the base revision and its own executable evidence. A hypothesis denotes an intended mechanism, a parent checkpoint, and supporting locations. If two hypotheses produce the exact same patch hash and full diff, the later hypothesis can point to the existing checkpoint through a transposition relation. This creates a search *graph*, rather than a tree whose identical states necessarily remain separate. Patch equality does not by itself establish environment equivalence or permit reusing an execution result.

3.2 Questions and versions

A question is represented conceptually as

$$q = \langle k, c, p, R, D, E, s, b, v \rangle, \quad (3)$$

where k is the action-related kind, c the claim, p the checkpoint, R the requirements, D the dependencies, E the supporting evidence, s the status, b a blocking reason, and v the evidence version. Questions and action records are obligation subtypes in the same graph, not an additional graph structure.

The implementation forms a stable question identifier from the checkpoint, kind, claim, requirement identifiers, and dependency identifiers. Its version hashes the patch identity, explicitly supplied relevant state, and the recorded revision and status of existing dependency and evidence nodes:

$$v(q, G) = \text{hash}(h_p, z_q, [(d, \text{rev}_G(d), \text{status}_G(d))]_{d \in D \cup E}). \quad (4)$$

Here z_q can contain semantic execution signatures, mechanical blockers, or a validation audit. This is a conservative recorded-state identity, not a minimal semantic dependence analysis. Because the full patch identity is included, unrelated edits can reopen a question. Conversely, unrecorded external state cannot be detected by this version alone.

A newly created question is **OPEN**. An observation may support or refute its claim, or establish a blocking reason. A replaced question can be superseded. When the recorded version changes, an existing question becomes open again. **BLOCKED** means that the controller lacks an admissible next step for that question under the current evidence and budget. It does not imply that its claim is true.

3.3 Authority is distinct from observation

An executable probe specifies its input, entrypoint, environment, observation adapter, and comparator. Executing the probe provides an observation; it does not create an oracle. Certification requires an expected behavior grounded in accepted public evidence, such as a public test or an explicit issue requirement with a traceable source binding. A model may propose a comparator, but cannot grant it authority by asserting that the comparator is correct.

This distinction matters for boundary challenges. A changed truthiness predicate suggests trying missing, empty, and nonempty values. It does not establish that all three inputs should be accepted. A challenge with an input but no grounded expectation remains exploratory. Its execution may refine localization or reveal a blocker, while leaving the corresponding specification question

unresolved. Similarly, syntax checks and imports are mechanical prerequisites, not substitutes for executing the required target behavior.

4 Demand-Driven Repair and Falsification

4.1 From evidence gaps to actions

The controller first attempts deterministic requirement extraction, source discovery, and available reproduction construction. It then generates an initial patch with bounded source context. After that patch, evidence recovery is reconsidered when executable target evidence or validation coverage is missing. This ordering avoids paying a model to orchestrate execution steps that already have a known implementation, while retaining model-based recovery for unresolved entrypoints or probe constructions.

The scheduler derives eligible actions from every open checkpoint in the graph. Its current priority order places mechanical repair first; missing validation and locked-success regression next; final review and eligible exploratory probes next; missing evidence recovery next; other stable failures next; and bounded graph expansion last. Preconditions resolve overlaps: a checkpoint without observations first requires validation, and an exhausted repair frontier may instead request expansion. This is a concrete heuristic policy, not an estimate of expected information gain.

Within a priority class, selection is lexicographic. For action a at checkpoint p , the ordering key is

$$\rho_G(a) = (\pi(a), -\hat{s}_G(p, a), -|O_a|, v_G(p), \hat{\tau}_G(a), \text{id}(a)), \quad (5)$$

where π is the priority class, $\hat{s}$ the executable-evidence score vector, O_a the associated obligations, v the visit count, and $\hat{\tau}$ the estimated duration. The vector is negated componentwise so that stronger evidence sorts first. For expansion actions, the score may include best-descendant potential; validation and certification use the checkpoint's own evidence. Duration is the median of recorded durations for the action kind when available, with a validation-specific estimate from recorded runs as a fallback; unknown cost sorts last. No model call is used to compute this ordering.

Algorithm 1 specifies the outer control policy. $\mathcal{A}(G)$ contains eligible actions derived from the open-question frontier, subject to execution and search preconditions. An action key $\kappa_G(a)$ combines its question, kind, and relevant evidence version; $\mathcal{H}(G)$ is the set of recorded attempted keys. Both are graph views, not external search state. DISPATCH performs the selected operation with the remaining budget and returns its outcome and measured usage. INTEGRATE records observations, patch provenance, question outcomes, and exhausted actions, including blocked executions. The production paths for recovery, validation, and generation implement this abstraction separately; it does not imply a universal interception gate for every internal model tool call.

4.2 Repair and targeted falsification

For a stable failure, localization ranks causal cuts using failure frames, executed paths, relevant value flow, and changed source. A cut provides real source spans and observations, rather than only graph identifiers. The repair request identifies the parent patch, the failed command, expected and actual observations, the proposed mechanism, and successes that must be preserved. Each candidate starts from its specified parent; its output is a complete base-to-child diff.

Falsification addresses a remaining claim about that candidate. The graph connects changed predicates and value-flow paths to boundary input recipes and affected checkpoints. Only executable, authority-grounded challenges can strengthen or refute certification. Exploratory probes are separately bounded. This makes the falsification role operationally different from a second model producing an unrestricted critique: a critique without an executable or evidence-recovery consequence does not establish a counterexample.

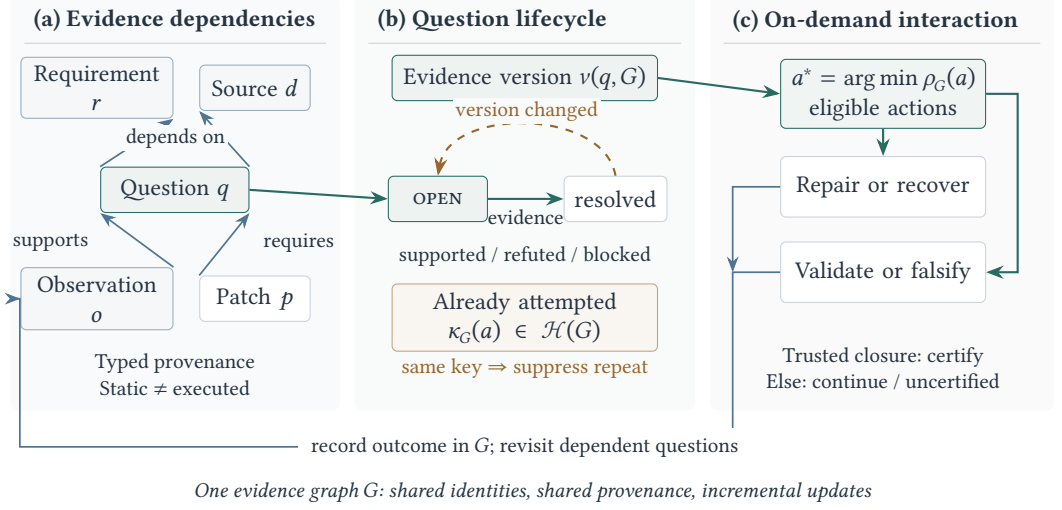


Fig. 1. Evidence-dependent interaction in ReachPatch. (a) A question links a checkpoint to its requirement, source, and execution evidence. (b) Recorded dependency changes reopen the question; an unchanged attempted action is suppressed under its reuse key. (c) The selected action returns evidence to the same graph. Certification requires trusted executable closure; blocked or exhausted evidence leads to an uncertified output, not success. Solid arrows denote evidence or control flow; the dashed feedback denotes version invalidation. The three panels are views of one state, not separate graph objects.

Rejected candidates remain in the graph with their failure evidence. The workspace can be restored to the recorded parent, and global selection can choose another checkpoint. A candidate that progresses on the target but regresses on a preservation obligation remains useful as a repair state, with passing target obligations locked. Retaining such a state supports repair of the regression without treating the candidate as final. Exact patch transpositions preserve alternative provenance without claiming a new distinct patch.

4.3 Three different forms of reuse

Action suppression. An action identity contains its question identity, kind, and evidence version. With suppression enabled, admission records an in-flight action before execution and rejects another attempt with the same stable identity. Execution then distinguishes completed work, retryable failure, and other failure. A transport error, timeout, or malformed JSON response permits one additional admission under the same identity, for at most two attempts in total; completed actions and exhausted retries remain suppressed. Changed evidence creates a new version rather than reusing the old result. This bounded policy neither guarantees exactly-once execution nor treats a failed interaction as evidence that the underlying question is resolved. Its false-suppression risk is an empirical safety question.

Evidence and context reuse. Source observations are keyed by path, content hash, and line range. A changed content hash marks previous source evidence stale. This identifies representational reuse; it need not eliminate physical file I/O when the caller has already computed the result. Within a model request, exact duplicate tool payloads can be replaced by shorter references only when the first full payload remains in that same request. Unique payloads and message envelopes

Algorithm 1 Evidence-dependent repair and falsification**Input:** Base repository R_0 , issue I , public evidence E_0 , budget B **Output:** Full base-to-selected diff Δ^* , decision z , evidence graph G

```

1:  $G \leftarrow \text{SEED}(R_0, I, E_0)$ 
2:  $(G, p_0, B) \leftarrow \text{BOOTSTRAP}(G, R_0, I, B)$   $\triangleright$  Recovery, initial patch, and evidence gaps
3: while  $\text{WITHINBUDGET}(B)$  do
4:   for all  $q \in \text{QUESTIONS}(G)$  do
5:      $v' \leftarrow v(q, G)$   $\triangleright$  Equation (4)
6:     if  $v' \neq q.v$  then
7:        $(q.s, q.v) \leftarrow (\text{OPEN}, v')$ 
8:     end if
9:   end for
10:   $\mathcal{C} \leftarrow \{p \in \text{EVALUATED}(G) \mid \text{CERTIFIED}(p, G)\}$ 
11:  if  $\mathcal{C} \neq \emptyset$  then
12:    return  $(\Delta(\text{SELECT}(\mathcal{C}, G)), \text{CERTIFIED}, G)$ 
13:  end if
14:   $\mathcal{A} \leftarrow \text{ELIGIBLEACTIONS}(G, B)$   $\triangleright$  Questions and execution prerequisites
15:  for all  $a \in \mathcal{A}$  such that  $\text{REUSEENABLED}(a, G)$  do
16:    if  $\kappa_G(a) \in \mathcal{H}(G)$  then
17:       $G \leftarrow \text{RECORDSUPPRESSION}(G, a); \mathcal{A} \leftarrow \mathcal{A} \setminus \{a\}$ 
18:    end if
19:  end for
20:  if  $\mathcal{A} = \emptyset$  then
21:    break  $\triangleright$  No admissible action; not evidence of success
22:  end if
23:   $a^* \leftarrow \arg \min_{a \in \mathcal{A}} \rho_G(a)$   $\triangleright$  Equation (5)
24:   $G \leftarrow \text{RECORDATTEMPT}(G, a^*, \kappa_G(a^*))$ 
25:   $(y, u) \leftarrow \text{DISPATCH}(a^*, R_0, G, B)$   $\triangleright$  Repair, recover, validate, probe, expand, or review
26:   $G \leftarrow \text{INTEGRATE}(G, a^*, y); B \leftarrow \text{DEBIT}(B, u)$ 
27: end while
28:  $p^* \leftarrow \text{BESTADMISSIBLEEVALUATED}(G)$ 
29:  $z \leftarrow \text{FINALSTATUS}(p^*, G, B)$   $\triangleright$  Certified only if executable closure holds
30: return  $(\Delta(p^*), z, G)$ 

```

remain intact. A later API call is not presumed to remember a previous request. Byte elision is recorded separately from provider-reported tokens.

Validation reuse. Execution caching is opt-in. A check must attest that it is isolated and deterministic, has no external state, and identifies its environment. Its key additionally includes clean and working tree hashes, the full check specification, an inherited-environment hash, execution backend metadata, adapter hashes, and the stability-run count. Only stable pass or fail results are cached. This is deliberately narrower than reusing results merely because two patches touch the same function. Its trust assumptions still require testing: an incorrect determinism attestation can invalidate reuse.

Table 1. Frozen 20-issue pilot: selected-generation-attempt cost only. Correctness non-inferiority is not established.

Metric	Fixed interaction	ReachPatch	Reduction
Model calls	439	323	26.4%
Total tokens	6,264,375	4,428,109	29.3%
Calls per issue	21.95	16.15	—
Tokens per issue	313,218.75	221,405.45	—

4.4 Stopping and selecting a patch

The controller distinguishes a certification decision from an economic stopping decision. Certification requires trusted executable target evidence, passing required obligations and locked successes, no confirmed authoritative preservation regression, and closure of applicable executable challenges. Budget exhaustion, blocked evidence, or an empty actionable frontier cannot substitute for these conditions.

Without certification, selection ranks admissible checkpoints using their executable evidence. The initial patch receives no preference simply because it was created first. A partial-progress checkpoint may be selected if it has no disqualifying blocker or confirmed preservation regression. Its output is explicitly uncertified. Thus, an inexpensive but evidence-limited outcome remains visible in evaluation without being counted as a successful repair.

5 Initial Empirical Evidence

We distinguish two fixed historical snapshots rather than merging different accounting boundaries. The first cutoff is September 20, 2026, 09:42 UTC, when 20 baseline issues had completed generation and could be matched to the historical method arm. The second is the completed 49-issue comparison at 12:58 UTC that day. The first is completion-selected; the second retains generation failures and archived attempts but still has uncontrolled retry and environment differences. Neither is a randomized experiment.

5.1 Protocol and accounting boundary

Both configurations use the same recorded production implementation, DeepSeek Flash, temperature zero, and disabled thinking. The client does not send a seed; temperature zero is not a reproducibility guarantee. Per-issue caps are 160 model calls, one million tokens, and 3,600 seconds. The search configuration permits eight evaluated patch nodes, depth three, and a branch factor of one. Consequently, this pilot does not evaluate the benefit of broad sibling search.

The baseline retains the unified graph and repair controller but disables evidence reuse and demand-driven interaction. It invokes a fixed post-initial-patch recovery stage with at most six agent turns. The method enables the two efficiency switches. Thus, this is a same-code joint ablation, not a bare-model baseline, an AgentDiet comparison, or an external state-of-the-art comparison. The two switches are confounded in this pilot.

The published cost boundary includes the selected successful *generation attempt* for each issue. “Successful” here means that the run produced its retained generation artifact, not that its patch passed hidden tests. Archived failed attempts are excluded. The historical arm has unequal retry histories and missing usage artifacts, so the observed reduction cannot be interpreted as an all-attempt cost reduction. Environmental load and provider drift were not randomized across arms.

Table 2. Completed historical 49-issue audit, including all recorded attempts. Missing usage makes token totals lower bounds, not complete bills. These arms have unequal retry histories and are not a prospective causal comparison.

Metric	Fixed interaction	ReachPatch
Recorded model calls	1,183	1,661
Reported tokens	16,349,466	21,453,024
Attempts without budget artifacts	0	50
Requests without token usage	4	0
Officially resolved	13	9
Officially unresolved	31	30
Generation failures	5	0
Evaluation errors	0	10

Table 1 computes each reduction as $1 - \sum_i C_i^{\text{method}} / \sum_i C_i^{\text{baseline}}$. This ratio of totals is not the mean of per-issue percentage reductions. No confidence interval over this selected cohort would remove its completion or retry selection bias. The table supports a narrower observation: retained attempts for these matched issues consumed fewer recorded calls and tokens.

5.2 Does the saving survive all-attempt accounting?

The larger audit reverses the direction of the descriptive cost contrast (Table 2). Recorded calls increase from 1,183 to 1,661 and reported tokens from 16,349,466 to 21,453,024: increases of 40.4% and 31.2%, respectively. Fifty method-arm attempts lack budget artifacts, while four baseline requests lack reported usage. The comparison is therefore incomplete on both sides; the missing method attempts cannot legitimately be assigned zero cost. In particular, the earlier 29.3% figure cannot support a claim about total workflow savings.

The completed audit records 13 resolved baseline issues and nine resolved method issues, with ten method evaluation errors. Treating those errors as unknown gives a method resolution interval of $[9/49, 19/49]$, compared with $13/49$ for the baseline including its five generation failures. Among the 39 pairs with known outcomes, there are two method wins and three losses. This conditional subset does not establish non-inferiority on the registered cohort. Neither a significance test on the selected subset nor an optimistic treatment of evaluation errors resolves the missing-outcome problem.

5.3 Does the trace support the proposed explanation?

A separate 49-issue method-arm report records 879 model calls and 12,049,596 tokens within its selected-attempt boundary. Of these calls, 544 (61.9%) belong to target recovery. The instrumentation records 132 source-read reuse events, but zero context-elided bytes, zero validation-cache hits, and zero action-suppression events under its corresponding counter. These are mechanism-activation measurements, not estimates of causal savings.

In particular, the pilot cannot attribute its token reduction to context compaction or validation caching: neither records an activation in that report. It also does not establish that graph-based duplicate suppression caused the reduction. The more plausible hypothesis to test is that demand-driven scheduling omits recovery rounds that the fixed schedule performs. A factorial ablation and request-level attribution are required to distinguish that hypothesis from shorter unsuccessful runs. Counting a repeated tool name and arguments is insufficient: a repeated test after an edit may be necessary, and a repeated source request may still deliver useful context to a stateless model.

Table 3. Prospective public-fixture factorial. Pass counts are independent public acceptance, not official Resolved@1.

Arm	Runs	Public pass	Calls	Reported tokens	Unknown usage
F00	3	3	33	151,113	0
F01	3	3	33	150,953	0
F10	3	3	16	46,469	0
F11	3	3	15	43,080	0

5.4 What is known about repair effectiveness?

The fixed pilot snapshot contains no baseline official-harness outcome. A separate completed submission of the 49 method-arm patches reports 14 initial patches resolved with seven harness errors, and nine final patches resolved with eleven harness errors. These totals have different evaluable subsets. The 31 issues with completed evaluations on both sides have nine resolved initial patches and nine resolved final patches.

The five apparent lost resolutions have identical initial and final patch hashes and infrastructure errors in final evaluation. They should not be interpreted as five confirmed semantic regressions. Conversely, excluding all infrastructure failures and reporting only the common subset would not establish whole-cohort repair preservation. These results provide neither a validated loss nor evidence of an improvement attributable to the method. Moreover, the initial-versus-final comparison is not the required fixed-interaction-versus-demand-driven comparison.

The present evidence therefore motivates, rather than completes, an effectiveness–efficiency evaluation. A publishable positive claim requires resolving the evaluation failures, counting all attempts, and comparing independent sealed final patches under matched conditions. Unit tests or the controller’s internal certification status cannot fill that gap.

5.5 Prospective integration study

We ran a blocked scheduling-by-reuse factorial on one public Python fixture, with 3 independently generated runs per arm. Arm order was randomized within each repetition; two blocks could run concurrently. The target requires empty-list support while preserving first-element behavior and a TypeError for None. Independent public executions check both target and preservation twice on the baseline and selected patch. This is an integration experiment, not an estimate of repository-level resolution. The F11-versus-F00 ratio-of-totals token reduction is 71.5%. We do not compute an issue-level confidence interval from repetitions of a single fixture. These observations do not establish repair non-inferiority or effectiveness of each individual reuse mechanism. A non-significant capability difference would not establish equivalence.

The request-level traces narrow the mechanism supported by this experiment. F00 and F01 each execute 18 recovery requests, whereas F10 and F11 execute none; all four arms execute nine repair requests. Thus the F00–F11 call difference is entirely accounted for by omitted recovery dialogue. Source-reuse hits, action-suppression hits, validation-cache hits, and context-elided bytes are all zero in this fixture. Their incremental benefit is therefore not demonstrated. F10 makes one more initial-generation request than F11, but a difference without reuse activation cannot reasonably be attributed to reuse. The observed factorial interaction should not be interpreted as evidence of mechanism synergy.

Table 4. Prospective development factorial. Missing official outcomes are reported separately; reported tokens exclude unknown usage rather than imputing it as free.

Arm	Resolved	Errors	Calls	Reported tokens	Unknown usage
F00	7/30	0	323	3,419,018	7
F01	8/30	0	334	3,571,680	8
F10	8/30	0	270	2,702,283	8
F11	8/30	0	282	2,961,716	7

An earlier integration attempt exposed an environmental failure: loss of the source-search executable and misinterpretation of a visible test command removed preservation evidence, permitting an incorrect internal certificate. We stopped that version, retained its recorded cost separately, and reran all four arms after adding environment preflight and command-parsing checks. The new repeated runs and a deterministic regression test pass independent preservation checks. This establishes correction of the observed failure, not the absence of all premature-certification failures.

The subsequent repository-level development run exposed a different failure in the edit-retry path: a non-applicable patch triggered an undefined reference while preparing source feedback. Four of the first 14 completed cells failed through this handler. We stopped that implementation and retained all 16 started attempts, comprising 189 admitted requests and 2,152,814 reported tokens, separately from the replacement experiment. We also corrected a tool-level duplicate filter that remained active in the no-reuse arm. The replacement protocol preserves the sampled cohort, arm order, and resource ceilings, but regenerates every arm independently. These implementation checks are necessary for an interpretable ablation; neither the interrupted cohort nor the corrections themselves establish an effectiveness gain.

5.6 Prospective development-cohort results

We evaluated 30 exposed development issues using repository-stratified sampling and randomized arm order within issue blocks. All four arms used DeepSeek Flash, temperature zero, a 40-call/250,000-token admission ceiling, a 900-second case budget, and at most two repair revisions. Each cell received one whole-case attempt; generation failures remain in the denominator. Official tests were opened only after every registered cell had sealed its final patch. These are developmental, not held-out confirmatory, results. Incomplete accounting prevents a complete-cost contrast; the token totals must be read as lower bounds. These observations do not establish repair non-inferiority or effectiveness of each individual reuse mechanism. A non-significant capability difference would not establish equivalence.

6 Controlled Evaluation Design

This section specifies the study needed to test the method’s claims; it reports no unperformed measurements. The design separates three questions: whether repair ability is retained, which operations are avoided, and whether version-aware evidence is necessary for avoiding them safely.

6.1 Research questions and baselines

RQ1: Effectiveness–cost trade-off. Does ReachPatch reduce all-attempt model cost while remaining non-inferior in official Resolved@1? We would compare the same-code fixed-interaction controller, a strong reproducible repair agent with the same model and tools where feasible, and the closest applicable efficiency baseline, AgentDiet [8]. Published scores from different models or

budgets are contextual information, not matched measurements. A direct-model baseline needs a documented source-acquisition and patch-output protocol; a model name alone is not a baseline.

RQ2: Contribution of scheduling and reuse. A 2×2 factorial experiment independently enables demand-driven interaction (D) and evidence reuse (R): $(0, 0)$, $(1, 0)$, $(0, 1)$, and $(1, 1)$. All arms retain the same public evidence, patch generator, graph construction, and budget ceilings. This separates the main effects and their interaction. Follow-up ablations split action suppression, source reuse, exact context elision, and opt-in validation caching. Deterministic target recovery is tested separately from scheduling so that removing a redundant model executor is not mistaken for a graph contribution.

RQ3: Does evidence structure add value beyond memoization? A flat exact-key cache is a serious alternative, not a straw baseline. It must receive the same source, execution, and environment identities. One control retains identical dependency semantics in a relational representation; equivalence would show that storage as a graph is not itself the contribution. A second control omits dependency-based reopening while retaining exact source and patch identity. This tests the role of evidence dependency tracking. An intentionally stale-cache policy is suitable only for offline stress tests, not a recommended production comparator.

RQ4: When is stopping safe? We would inspect stopped and suppressed actions using an isolated continuation experiment. For a prespecified sample, resume the stopped state with a fixed additional allowance while holding the model and public information constant. Record whether continuation uncovers a trusted counterexample or produces a patch later accepted by held-out evaluation. These continuations are diagnostic experiments; they must not replace the sealed primary patch. They estimate missed opportunities under a finite continuation policy, not an oracle’s judgment that every omitted call was useless.

RQ5: Generality and interaction with prior methods. Repeat the main comparison across at least two model families and a held-out issue cohort that does not overlap development cases. A composition experiment with AgentDiet distinguishes avoiding interactions from compressing their histories. ExpeRepair’s experience memory [3] and ADI’s dynamic interface [7] motivate further transfer tests, but integrating them changes the agent and must be reported as such rather than presumed to be a drop-in equivalent configuration.

6.2 Metrics and analysis

The primary outcome is one sealed final patch per issue, evaluated by a fixed hidden harness using the benchmark’s containerized evaluation procedure [4]. All intended issues remain in the denominator, including generation failures. Infrastructure failures are reported separately, retried under a policy fixed before outcomes are inspected, and accompanied by missing-outcome bounds if unresolved. No hidden feedback is used to revise generated patches.

Cost includes prompt, completion, and any separately billed reasoning tokens for every stage and attempt. Cached and uncached prompt tokens are separated, and monetary cost uses a dated price schedule. Model calls include paid or attempted requests that fail before patch output. Report mean, median, and tail latency, as well as model, tool-execution, graph-maintenance, and environment-setup time. Cost per resolved issue uses total cohort cost divided by resolved issues, rather than averaging only successful trajectories.

Mechanism metrics include repeated action rate under an explicit identity, duplicate-only model rounds, questions reopened after evidence changes, reused evidence counts, context tokens actually removed, target-recovery success, executable challenge yield, and unresolved questions at

termination. Action novelty is not synonymous with action utility. Manually auditing a prespecified, stratified sample with independent annotators can distinguish useful repeated validation, redundant attempts, blocked infrastructure actions, and necessary new exploration.

For effectiveness, preregister δ and test a paired non-inferiority contrast; a nonsignificant superiority test is not evidence of equivalence. Determine sample size from expected baseline success and the paired discordance rate. Twenty or fifty issues are unlikely to support a tight margin. For cost, report paired differences and ratios of aggregate costs with issue-level bootstrap intervals, retaining all repeats of the same issue within a resample. Analyze repository heterogeneity separately because a few repositories do not justify asymptotic cluster claims. Report matched wins and losses as well as aggregate resolution rates.

A common set of budget ceilings establishes an equal-opportunity comparison. Additional preregistered budget levels establish a resolution–cost frontier: spending less is desirable only if the loss in repair ability stays within the chosen margin. Fixed-trajectory replay can measure exact token elision and cache eligibility, but cannot predict the repairs a model would generate after changed context. Therefore replay complements, rather than replaces, end-to-end randomized runs.

7 Related Work

Trajectory reduction and efficient editing. AgentDiet identifies useless, redundant, and expired trajectory information and invokes a separate reflection model on a sliding window [8]. Its evaluation explicitly includes reflection overhead and distinguishes input-token reduction from net monetary savings. This is the closest efficiency comparison. Our question is upstream: whether a recorded unresolved claim still requires an interaction. Our exact within-request elision is substantially narrower than AgentDiet’s semantic compression, and we do not claim novelty for removing repeated text. The two mechanisms may be complementary, which must be measured rather than assumed. Cascaded Code Editing instead separates edit-sketch generation from sketch application and trains smaller models for the latter [5]. It reduces the cost of producing edits, while our controller decides whether another repair-related action is warranted.

Procedural repair. Agentless uses a fixed repair pipeline [6]. It localizes likely edits, generates patches, and validates candidates without delegating every decision to an autonomous agent. This is an important counterpoint to adding control machinery: a simpler pipeline can already be cost-effective. Our proposed distinction is not deterministic orchestration alone, but explicit evidence-dependent action eligibility and reopening. A matched comparison against a simple pipeline is needed to establish whether that additional state is worthwhile.

Memory and dynamic evidence. ExpeRepair accumulates episodic demonstrations and semantic insights across issues and composes repair prompts from that experience [3]. Our evidence reuse is primarily within one issue and bound to recorded source and checkpoint state, not retrieval of analogous historical repairs. ADI packages function execution into Frame Lifetime Traces and exposes agent-oriented navigation [7]. It directly motivates measuring interaction granularity and useful dynamic evidence. ReachPatch does not claim equivalent tracing completeness; its contribution under investigation is the lifecycle of evidence questions and actions using available traces.

Repair and reproduction interaction. Cheng et al. study freeform, test-first, and test-last cogenation, along with test-aware patch selection [1]. Their analysis highlights fix–test interference and the distinction between plausible reproductions and reliable selection. Our repair–falsification loop likewise cannot treat a self-generated passing test as sufficient correctness evidence. The difference is an explicit demand condition for the next action and a provenance constraint on what a

challenge can establish. The industrial study concerns joint fix-and-test generation; our efficiency hypothesis concerns whether additional recovery or challenge work is needed for an unresolved question.

8 Limitations and Threats to Validity

Internal validity. The retained-attempt pilot jointly changes two policies, selects cases by completion time, excludes archived attempts, and compares historical runs under uncontrolled load. The completed historical audit exposes additional expenditure rather than confirming the apparent saving. The prospective implementation now distinguishes completed actions from bounded retries after transport failures and writes request admission before dispatch. These safeguards improve auditability but do not retroactively repair missing historical records. Whole-patch versioning remains conservative, and dependency registration still determines which questions reopen; dedicated stale-evidence stress tests remain necessary.

Construct validity. A repeat is not necessarily waste, and an exploratory probe is not an oracle. Provider token usage is not identical to billed cost or energy; context bytes are not tokens. Certification means closure of available executable obligations, not full specification correctness. Hidden harnesses are themselves incomplete correctness checks. A stronger final study should combine official evaluation with blinded inspection of changed-outcome patches and newly found regressions.

External validity. The current pilot covers a small repository-issue cohort with one model and Python-oriented source and trace analysis. It cannot establish cross-model, cross-language, or industrial generality. The full public benchmark also has a contamination risk; a temporally separated evaluation reduces, but cannot prove the absence of, training overlap. Development issues must not be reintroduced as unseen test data, and per-repository outcomes must remain visible.

Novelty and implementation fidelity. The graph representation alone is not an algorithmic advance over a relational store with equivalent semantics. The research claim must rest on demonstrably useful question lifecycle and action scheduling. Likewise, dormant mechanisms cannot explain observed gains: the recorded zero-hit compaction and validation caches should be treated as unvalidated components. A full-code audit and protocol freeze are prerequisites to a final evaluation, not facts established by the pilot.

9 Conclusion

ReachPatch investigates a different unit of control for repair agents: a versioned unresolved evidence question. Requirements, source, execution observations, and patch states share one graph, allowing deterministic action selection to distinguish a new investigation from an unchanged attempted one. The historical audit shows why a positive retained-attempt comparison is insufficient: including archived attempts reverses the recorded cost contrast. Preserved repair effectiveness and net savings are not yet established. A controlled factorial study, complete request accounting, and a matched comparison with trajectory reduction are necessary to determine whether question-driven interaction provides an effectiveness–cost advantage.

AI Assistance Disclosure

An AI coding assistant was used to inspect implementation artifacts, retrieve literature, and draft manuscript text, equations, and the evaluation plan. The human authors must verify all claims, citations, and results before submission and take responsibility for the final work. No AI system is listed as an author.

References

- [1] Runxiang Cheng, Michele Tufano, José Cambronero, Renyao Wei, Sherry Shi, Grant Uy, Pat Rondon, and Franjo Ivančić. 2026. Dynamic Cogeneration of Bug Reproduction Test in Agentic Program Repair. In *Companion Proceedings of the 34th ACM International Conference on the Foundations of Software Engineering (FSE Companion '26)*. Association for Computing Machinery, New York, NY, USA, 11 pages. doi:10.1145/3803437.3805237 Industry track; author manuscript arXiv:2601.19066v3.
- [2] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-world GitHub Issues?. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=VTF8yNQM66>
- [3] Fangwen Mu, Junjie Wang, Lin Shi, Song Wang, Shoubin Li, and Qing Wang. 2026. ExpeRepair: Dual-Memory Enhanced LLM-Based Repository-Level Program Repair. *Proceedings of the ACM on Software Engineering* 3, FSE, Article FSE174 (2026), 23 pages. doi:10.1145/3808181
- [4] SWE-bench Team. 2026. SWE-bench: Official Documentation. <https://www.swebench.com/SWE-bench/>
- [5] Chaozheng Wang, Zezhou Yang, Shuzheng Gao, Cuiyun Gao, Zongjie Li, Yichen Li, Ting Peng, Hailiang Huang, Yuetang Deng, and Michael R. Lyu. 2026. Cascaded Code Editing: Large-Small Model Collaboration for Effective and Efficient Code Editing. *Proceedings of the ACM on Software Engineering* 3, FSE, Article FSE094 (2026), 23 pages. doi:10.1145/3808101
- [6] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2025. Demystifying LLM-Based Software Engineering Agents. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 801–824. doi:10.1145/3715754 FSE research track; author manuscript arXiv:2407.01489v2.
- [7] Jiahong Xiang, Xiaoyang Xu, Xiaopan Chu, Hongliang Tian, and Yuqun Zhang. 2026. Empowering Autonomous Debugging Agents with Efficient Dynamic Analysis. *Proceedings of the ACM on Software Engineering* 3, FSE, Article FSE031 (2026), 22 pages. doi:10.1145/3797126
- [8] Yuan-An Xiao, Pengfei Gao, Chao Peng, and Yingfei Xiong. 2026. Reducing Cost of LLM Agents with Trajectory Reduction. *Proceedings of the ACM on Software Engineering* 3, FSE, Article FSE056 (2026), 22 pages. doi:10.1145/3797084